



南開大學
Nankai University

计算机学院
大数据计算及应用实验报告

Track 2: 电影评分预测系统
(Prompt Engineering)

姓名：王旭
学号：2312166
专业：计算机科学与技术

2026 年 6 月 28 日

目录

1 摘要	2
2 实验目的与任务说明	2
2.1 任务背景与目标	2
2.2 数据集说明	2
2.3 评分体系与优化优先级	2
3 训练集数据分析	3
3.1 评分分布与用户尺度差异	3
3.2 冷启动与字段命名差异	3
3.3 相似电影 API 的局限	3
4 实验评测流程与设计	4
4.1 本地调试与网站评测	4
4.2 Prompt 整体架构	4
5 Prompt 迭代历程与成败的分析体会	4
5.1 迭代总览	4
5.2 v1-v2: 短输出与冷启动参考	5
5.3 v3: 硬约束“接近均分”导致跌分	5
5.4 v7-v8: custom_similar 与 similar 减为 1 条	5
5.5 v9-v12: 在 v8 上继续改均未见效	6
5.6 v13: 冷启动去掉“参考均分”一行	6
6 最终方案 v13 的设计要点	6
6.1 非冷启动分支	6
6.2 冷启动分支	6
7 评分结果	7
7.1 评分集结果 (#1495、#1497)	7
7.2 评测平台异常与恢复	7
8 总结与展望	8
8.1 主要结论	8
8.2 不足与后续工作	8
9 github 仓库	8

课程实验报告：Track 2 电影评分预测系统

1 摘要

本实验在课程给定的豆瓣数据集上，通过 **Prompt Engineering** 引导 **GLM-4.5-Air** 预测用户对目标电影的评分（1-5 整数）。核心为 `prompt.py` 中的 `generate_prompt(ctx)` 函数：利用 `ctx` 系列 API 构造 System/User Prompt，并强制模型仅输出一行 `[Result:X]` 以控制 Token 成本。

实验经历 **v1**→**v13** 共十三轮迭代。关键成功路径包括：(1) 输出格式约束使 100 条样本输出 Token 恒为 **600**；(2) 冷启动多样跨用户参考与用户尺度校准；(3) 自定义标签匹配 `custom_similar` 替代的平台 `get_similar_movies`；(4) 最相关历史由 2 条减至 1 条；(5) 冷启动 User Prompt 中去除“参考均分约 X.X”的约束。失败案例包括 v3 的硬统计约束、v9-v12 在 v8 基础上的堆叠改动。

最终的定稿版本 v13 在测试集 **#1281** 上得分 **98.58**；评分集两次提交分别得分 **93.39 (#1495)**、**94.30 (#1497)**。各次的全量输出 Token 均为 **600**。

关键词： Prompt Engineering GLM-4.5-Air 冷启动 Token 效率 推荐收益率

2 实验目的与任务说明

2.1 任务背景与目标

本任务模拟推荐系统中“用 LLM 预测用户对未观影电影的评分”的场景。模型固定为 **glm-4.5-air**，配置 `temperature=0`、关闭思考模式、不传 `tools`，以保证输出稳定，通过包含 `generate_prompt` 的 Python 脚本来进行实际预测。

任务目标不仅是降低 MAE/RMSE，更要在**准确率**与**Token 成本**之间取得平衡——我们的实验指导书中将“推荐收益率”设为单项最高权重（50%），来模拟真实推荐中精度与边际成本的关系。

2.2 数据集说明

数据集	规模	说明
训练集	2000 条	200 用户 × 10 条，本地可见，用于分析与 Prompt 设计
测试集	100 条	50 用户 × 2 条，含冷启动用户，用于网站测试
评分集	100 条	与测试集分布类似，用于网站评分
电影库	27146 部	含冷启动电影

表 1: 数据集说明

我们的训练集**无冷启动样本**（每用户固定 10 条历史），而测试/评分集包含 `is_cold_start` 用户，因此对于冷启动策略，我们无法从训练集直接验证，必须结合单样本测试与实际评测来迭代。

2.3 评分体系与优化优先级

指导书将满分 120 分截断至 100 分，六项指标权重如下：

指标	权重	含义
推荐收益率	50%	模拟每次推荐的利润 $(R - C)/C$
±1.0 准确率	30%	预测与真实分差 ≤ 1 的占比
精准命中率	10%	误差 < 0.5 (即完全命中整数分)
MAE / RMSE	各 10%	传统回归误差
Token 效率	10%	平均每条输入 + 输出 Token 数

表 2: 评测分项权重

收益率规则中：精准命中收益 **10 元/条**，±1 分内仅 **2 元/条**，误差 > 1 分只有 **0.1 元/条**；每次推荐有 **1 元** 固定成本及 Token 费用。因此**减少输出 Token** 与**提高精准命中率**同样重要，而不能单纯追求 MAE 最小化。

3 训练集数据分析

我们首先对训练集 `student/data/train.json` (200 用户 $\times$ 10 条) 与 `movies_info.json` 进行统计，先从可见的数据归纳可以写入 Prompt 的规律，并评估各 `ctx` API 的重要性和 Token 成本。

3.1 评分分布与用户尺度差异

全局 2000 条评论均分 **3.65**，4 分占比最高 (38.8%)。但用户间差异显著：

- **29%** 用户均分 ≥ 4.0 (偏宽松)，**11%** 用户均分 ≤ 3.0 (偏严格)；
- 单用户 10 条评分极差平均 **2.5** 分，多数用户的评价既有高分也有低分；

因此，Prompt 里需要写清**该用户自己的打分尺度** (`get_user_stats()` 的 avg、min、max、distribution)，不能默认所有人都按 4 分在评。

3.2 冷启动与字段命名差异

测试/评分集包含冷启动用户，比如我们进行单样本验证时所产生的结果：用户 `U_183aba26`、目标片《硬汉》。我们的训练集中没有这类样本，需要独立设计冷启动分支。

另外，历史记录使用的是 `movie_name`，而 `movies_info` 用的则是 `name`。最开始我们的 v0 版本曾因未兼容导致单样本测试时，片名显示“未知”，因此我们定稿中使用：

片名字段兼容 (`movie_name / name`)

```
1 def movie_title(m):
2     return m.get('name') or m.get('movie_name') or '未知'
```

3.3 相似电影 API 的局限

平台提供的 API `get_similar_movies` 基于 `tags.split(',')` 求交集，但训练集的 tags 存在 Python 列表串与“逗号 + 数字 ID”两种格式，约 **60%** 的样本交集为 0。因此从 v7 起我们改用了自实现的 `custom_similar`：先把 tags 解析成统一列表，再按与目标片标签的交集数量、是否同导演综合打分并排序。v7 时仍取前 2 条最相关历史；v8/v13 改为只保留**排名第 1 的那一条**，因为第二条往往只是弱相关，容易和最近评分、极值样例的趋势相冲突，实测去掉后精准命中明显回升。

4 实验评测流程与设计

4.1 本地调试与网站评测

1. 验证代码（不限次）：检查提交格式与安全性；
2. 单样本测试（不限次）：检查 Prompt 长度、LLM 响应、解析结果；
3. 测试集评测（每天 3 次）：100 条完整评测，记录分项得分；
4. 评分集评测（共 2 次）：100 条样本测试，计入最终客观成绩。

4.2 Prompt 整体架构

我们的最终定稿 v13 的 Prompt 由 System 与 User 两部分构成，逻辑如图 4.1 所示。

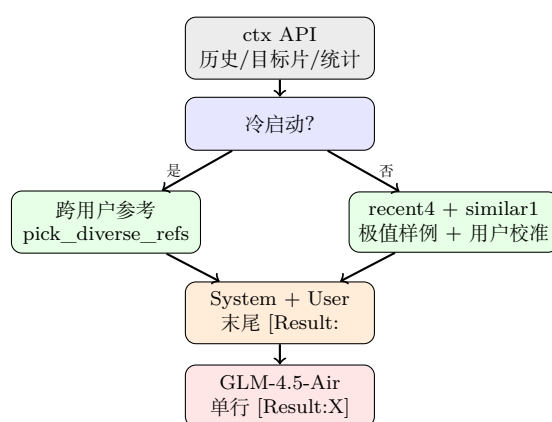


图 4.1: 定稿 v13 Prompt 的逻辑流程

5 Prompt 迭代历程与成败的分析体会

5.1 迭代总览

表 3 汇总 v1–v13 的测试集评测历程与结果。图 5.2 展示最终得分随版本变化的趋势。

版本	主要改动	精准命中率%	$\pm 1.0\%$	输出 Tok	得分
v1	短输出约束；冷启动分支；movie_name 修复	43	90	600	94.48
v2	多样冷启动；用户校准；compact 格式	47	86	600	95.47
v3	加上“接近均分 ± 1 ”的硬约束	41	88	600	93.52
v4	similar 3 条；软约束（可参考均分，不强制相差 ± 1 ）	40	93	600	95.20
v5	recent4 + 众数 + 极值样例附 tags	43	91	600	95.52
v6	标签偏好 + 常态参照的影片	47	88	600	95.04
v7	v5 + custom_similar	49	88	644	96.54
v8	最相关历史 2 条 $\rightarrow$ 1 条	52	90	600	98.26
v9–v12	过滤/降权/年份/冷启动重选（均失败）	47–50	88–90	600–644	96.09–96.90
v13	去掉冷启动的“参考均分”	51	91	600	98.58

表 3: Prompt 版本迭代摘要（测试集测试结果）

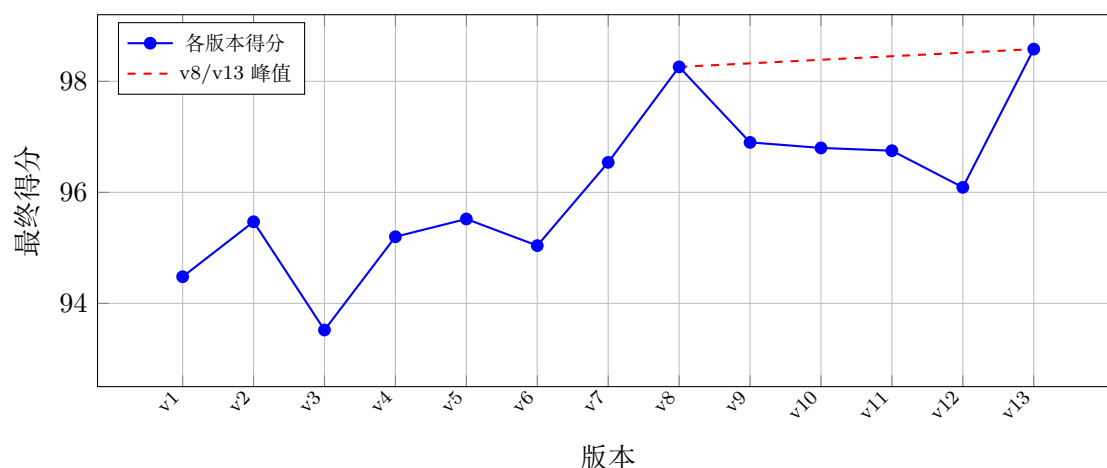


图 5.2: 测试集最终得分随 Prompt 版本的变化 (v3 与 v9-v12 为失败的尝试过程)

5.2 v1-v2: 短输出与冷启动参考

v1 在 System Prompt 中强制“回复有且仅有一行 [Result:X]”, User Prompt 末尾预填 [Result:], 使 100 条全量输出 Token 恒为 600 (约 6 tok/条)。这是最直观并且零成本的最大提分项, 防止了 GLM 大模型默认的长篇分析, 从而输出一大堆无关的输出 Token。

v2 针对冷启动: 用 get_random_user_history 采样并按 2/3/4/5 分档挑选参考, 避免跨用户参考全为 4 分导致系统性偏高; 同时加入用户 strict/lenient 的严格或宽松的校准与 compact 格式, 输入 Token 下降了大约 16%, 精准命中从 43% 升至 47%。

核心输出约束片段如下:

输出格式约束 (System/User Prompt 核心片段)

```
1 system_prompt += """
2 输出要求 (必须严格遵守):
3 - 不要输出分析、解释、理由或任何其他文字
4 - 回复有且仅有一行, 格式为: [Result:X] (X为1-5的整数) """
5
6 user_prompt = f"""...
7 [Result: """
```

5.3 v3: 硬约束“接近均分”导致跌分

v3 在 v2 基础上增加“无充分理由时预测应与均分接近 (通常相差不超过 1)”的硬约束。结果综合分 93.52 (低于 v2): 精准命中 47%→41% (-6%), 收益减少约 44 元。

这一版给我们的体会是: 均分、众数这类数字可以放进 Prompt 当参考, 但不宜写成“必须跟着均分走”。v3 之后我们基本不再加强制服从统计量的句子; 后面 v13 去掉冷启动里“参考均分约 X.X”那一行后能再涨分, 也和 v3 的教训是同一类问题。

5.4 v7-v8: custom_similar 与 similar 减为 1 条

v6 在 v5 上又加了“标签偏好摘要 + 常态参照影片”, RMSE 变差、综合分降到 95.04。v7 回退到 v5 框架, 只把 similar 换成 custom_similar, 得分 96.54。v8 再把最相关历史从 2 条减到 1 条, 减少一条弱相关片的干扰, 测试集 #953 得分 98.26 (精准 52%)。

custom_similar 调用 (v8/v13: 最相关历史仅 1 条)

```
1 def custom_similar(history, target_tags, target_director, n=2):
2     # 标签交集 * 10 + 导演匹配 + 时间序微扰
3     ...
4 similar_pairs = custom_similar(ctx.user_history, target_tags_list,
5                                target_director, 1) # v8/v13: n=1
```

5.5 v9–v12: 在 v8 上继续改均未见效

v8 定稿之后我们又试了四次 (v9–v12)，本来还想把分数往 99 分推，结果都在 96.09–96.90 之间，没有一次超过 v8。改动包括：similar 过滤 (v9)、泛标签降权 (v10)、目标片加年份 (v11)、冷启动参考重选 (v12)。

这几个版本有一个共同规律，**提示我们**：在已收敛的 v8 框架上**继续加规则**，往往损害精准命中或 RMSE，而收益率 50% 权重使小幅误差的代价放大。

5.6 v13: 冷启动去掉“参考均分”一行

v13 相对 v8 **仅修改了一处**：冷启动 User Prompt 中去掉了“(参考均分约 X.X)”提示，保留多样跨用户参考 (2/3/4/5 分各一条) 与标签相关度排序。

测试集 #1281 结果：最终得分 **98.58**， ± 1.0 升至 **91%**，RMSE 降至 **0.90**，输出 Token 仍为 **600**。去掉 Prompt 里写死的参考均分之后，模型不再一直往“跨用户样本的平均分”上靠，冷启动预测反而更稳一些。

6 最终方案 v13 的设计要点

6.1 非冷启动分支

1. **System 用户校准**：根据 `avg` 分为偏严格 / 偏宽松 / 居中三种说明；
2. **User 画像**：均分、区间、众数、分布、最近均分；
3. **recent 4 条 + custom_similar 1 条** (带命中标签)；
4. **高低分极值样例**各 1 条 (用户历史里最高/最低分片)，附 tags 与 28 字短评；
5. **目标影片**：名称、导演、类型 (前 5 标签)、简介 (180 字截断)。

6.2 冷启动分支

1. `build_cold_ref_pool`：多次 `get_random_user_history` 去重采样；
2. `pick_diverse_refs`：按标签相关度排序，优先选取 2/3/4/5 分各一条；
3. System 补充“优先参考标签接近的示例，避免系统性偏高”；
4. **相对 v8 版本的改进**：不再写入“参考均分约 X.X”。

7 评分结果

7.1 评分集结果（#1495、#1497）

定稿 v13 在测试集 #1281 上得分 98.58；对于评分集的评测，我们按照 v13 版本进行了两次评分集的测试，输出 Token 均为 600，状态均为 success，取较高分 **#1497 = 94.30**。冷启动单样本（《硬汉》）输出 `[Result:3]`，格式解析正常。

指标	测试集 #1281	评分集 #1495	评分集 #1497
最终得分	98.58	93.39	94.30
MAE	0.59	0.7475	0.7273
RMSE	0.900	1.0731	1.0347
精准命中	51%	42.42%	41.41%
±1.0 准确率	91%	85.86%	88.89%
输出 Token	600	600	600
总收益	590.90 元	507.40 元	505.10 元

表 4: 测试集与评分集对比（均为 v13；客观成绩取 #1497）

结果解读：评分集与测试集样本不同，98.58 与 94.30 之间的差距主要来自**精准命中率**降低（51%→约 42%/41%），收益率分项因此低于测试集。同一份 v13 连测两次，#1497 比 #1495 高 **0.91** 分：精准命中略降 1 条，但 ±1.0 从 85.86% 升到 88.89%（该项权重 30%），MAE/RMSE 也略好。

7.2 评测平台异常与恢复

我们在 6 月 24 日进行测试时，发现同一份 v8 代码进行两次复测都出现得分骤降的现象（**#1149：88.72**，**#1210：80.54**），并且输出 Token 膨胀至 **1304/2712**，耗时都在 20–30 分钟。当输出 Token 600 才能说明测试环境正常；并且我们当时进行了单样本测试，输出 LLM 响应仍为一行 `[Result:3]`，说明 Prompt 没有问题而是测试的全量链路异常。

6 月 25 日平台恢复后：v8 复测 #1274 得分 96.39（输出 Token 恢复正常为 600）；v13 测试 #1281 得分 98.58（输出 600）。

评测 ID	版本	得分	输出 Token	状态
#953	v8	98.26	600	作为有效基线
#1149	v8	88.72	1304	平台异常
#1210	v8	80.54	2712	异常加剧
#1274	v8 复测	96.39	600	恢复确认
#1281	v13	98.58	600	v13 的最终测试集
#1495	v13	93.39	600	评分集第 1 次
#1497	v13	94.30	600	评分集第 2 次

表 5: 关键评测记录（含平台异常、测试集与评分集）

8 总结与展望

8.1 主要结论

1. 本任务的 Prompt 工程是在**准确率与 Token/收益约束下的联合优化**，收益率 50% 权重决定了“短输出 + 精准命中”优先于单纯降低 MAE。
2. **做减法**（比如我们的 v7 去掉 v6 堆叠、v8 减 similar、v13 删参考均分行）比继续在 v8 上加规则更管用；v9–v12 基本说明了对于 v8 版本后面能改的空间不大。
3. 对不可控的 LLM 输出响应，应该把自由度限制在唯一输出结果 `[Result:X]` 上；输出 Token ≈ 600 可说明评测环境正常。
4. 定稿 **v13** 的测评结果：测试集 **98.58** (#1281)；评分集 **94.30** (#1497)；两次评分集 93.39 / 94.30，取较高者。

8.2 不足与后续工作

- 评分集精准命中（约 41–42%）低于测试集（51%），但 ± 1.0 仍达 86–89%；同一 v13 版本连测两次相差 0.91 分，说明边界样本上仍有波动空间；
- LLM 非确定性 & 测试/评分样本差异，会使同 Prompt 全量得分相差数分（比如我们的测试集 #953 与 #1274 对比也差 1.87 分）；

9 github 仓库

代码详见：<https://github.com/Wangxuueuuu/Big-Data-Computing-and-Applications>